

outimp的解题思路

题目信息

题目名	类型	难度
outimp	DS	入门

FLAG

- 动态flag

知识点

1. 数据脱敏

解题步骤

1、拿到附件的data.csv后打开发现是各种数据信息，然后按照附件提供数据脱敏规范文档里的脱敏规则编写以下poc.py，实现数据脱敏

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
数据脱敏题目 - 解题脚本
按照数据脱敏规范文档对敏感数据进行脱敏处理
"""

import csv
import re
from datetime import datetime

def mask_name(name):
    """
    姓名脱敏
    规则：
    - 两字姓名：脱敏第二个字
    - 三字姓名：脱敏中间一个字
    - 四字姓名：脱敏中间两个字
    - 其他长度姓名：保留首尾字符，中间用星号(*)替换
    """
    if not name or len(name) < 2:
        return name

    name_len = len(name)

    if name_len == 2:
        # 两字姓名：脱敏第二个字
        return name[0] + '*'
    elif name_len == 3:
        # 三字姓名：脱敏中间一个字
```

```

        return name[0] + '*' + name[2]
    elif name_len == 4:
        # 四字姓名：脱敏中间两个字
        return name[0] + '**' + name[3]
    else:
        # 其他长度姓名：保留首尾字符，中间用星号替换
        middle_length = name_len - 2
        return name[0] + '*' * middle_length + name[-1]

def mask_id_card(id_card):
    """
    身份证号脱敏
    规则：保留前6位（地区码）和后4位（校验码等），中间用星号(*)替换
    """
    if not id_card or len(id_card) < 10:
        return id_card

    # 计算中间需要替换的位数
    middle_length = len(id_card) - 10 # 总长度 - 前6位 - 后4位
    masked_middle = '*' * middle_length

    return id_card[:6] + masked_middle + id_card[-4:]

def mask_mobile(mobile):
    """
    手机号脱敏
    规则：保留前3位和后4位，中间4位用星号(*)替换
    """
    if not mobile or len(mobile) != 11:
        return mobile

    return mobile[:3] + '****' + mobile[-4:]

def mask_birth_date(birth_date):
    """
    出生日期脱敏
    规则：保留年份和月份，日期用星号(*)替换
    格式：YYYY-MM-DD → YYYY-MM-**
    """
    if not birth_date:
        return birth_date

    # 匹配日期格式 YYYY-MM-DD
    date_pattern = r'^(\d{4})-(\d{2})-(\d{2})$'
    match = re.match(date_pattern, birth_date)

    if match:
        year, month, day = match.groups()
        return f"{year}-{month}-**"

    return birth_date

```

```

def mask_bank_card(bank_card):
    """
    银行卡号脱敏
    规则：保留前4位和后4位，中间位数用星号(*)替换
    """
    if not bank_card or len(bank_card) < 8:
        return bank_card

    # 计算中间需要替换的位数
    middle_length = len(bank_card) - 8
    masked_middle = '*' * middle_length

    return bank_card[:4] + masked_middle + bank_card[-4:]

def process_data(input_file, output_file):
    """
    处理数据文件，对敏感字段进行脱敏
    """
    processed_count = 0

    try:
        with open(input_file, 'r', encoding='utf-8') as infile, \
            open(output_file, 'w', newline='', encoding='utf-8') as outfile:

            reader = csv.DictReader(infile)
            fieldnames = reader.fieldnames
            writer = csv.DictWriter(outfile, fieldnames=fieldnames)

            # 写入表头
            writer.writeheader()

            # 处理每一行数据
            for row in reader:
                # 脱敏处理
                if '姓名' in row:
                    row['姓名'] = mask_name(row['姓名'])

                if '身份证' in row:
                    row['身份证'] = mask_id_card(row['身份证'])

                if '手机号' in row:
                    row['手机号'] = mask_mobile(row['手机号'])

                if '出生日期' in row:
                    row['出生日期'] = mask_birth_date(row['出生日期'])

                if '银行卡号' in row:
                    row['银行卡号'] = mask_bank_card(row['银行卡号'])

            # 写入脱敏后的数据
            writer.writerow(row)
            processed_count += 1
    
```

```

        # 进度提示
        if processed_count % 1000 == 0:
            print(f"已处理 {processed_count} 条记录")

    except FileNotFoundError:
        print(f"错误：找不到输入文件 {input_file}")
        return False
    except Exception as e:
        print(f"处理过程中发生错误：{e}")
        return False

    print(f"\n脱敏处理完成！")
    print(f"总计处理：{processed_count} 条记录")
    print(f"输出文件：{output_file}")

    return True

def validate_masking(input_file, output_file):
    """
    验证脱敏效果
    """
    print("\n=== 脱敏效果验证 ===")

    try:
        with open(input_file, 'r', encoding='utf-8') as infile, \
            open(output_file, 'r', encoding='utf-8') as outfile:

            original_reader = csv.DictReader(infile)
            masked_reader = csv.DictReader(outfile)

            # 检查前5条记录
            print("\n原始数据 vs 脱敏数据对比（前5条）：")
            print("-" * 120)

            for i, (orig_row, masked_row) in enumerate(zip(original_reader,
masked_reader)):
                if i >= 5:
                    break

                print(f"\n记录 {i+1}:")
                print(f"姓名: {orig_row.get('姓名', '')} → {masked_row.get('姓名',
''}})")
                print(f"身份证: {orig_row.get('身份证', '')} → {masked_row.get('身份
证', '')}}")
                print(f"手机号: {orig_row.get('手机号', '')} → {masked_row.get('手机
号', '')}}")
                print(f"出生日期: {orig_row.get('出生日期', '')} → {masked_row.get('出
生日期', '')}}")
                print(f"银行卡号: {orig_row.get('银行卡号', '')} → {masked_row.get('银
行卡号', '')}}")

```

```

except Exception as e:
    print(f"验证过程中发生错误：{e}")

def main():
    """
    主函数
    """
    print("=== 数据脱敏处理程序 ===")
    print("按照数据脱敏规范文档进行敏感数据脱敏")

    input_file = 'data.csv'
    output_file = 'answer.csv'

    print(f"\n输入文件：{input_file}")
    print(f"输出文件：{output_file}")

    # 执行脱敏处理
    success = process_data(input_file, output_file)

    if success:
        # 验证脱敏效果
        validate_masking(input_file, output_file)

        print("\n=== 脱敏规则说明 ===")
        print("1. 姓名：保留姓氏，名字用*替换")
        print("2. 身份证：保留前6位和后4位，中间8位用*替换")
        print("3. 手机号：保留前3位和后4位，中间4位用*替换")
        print("4. 出生日期：保留年月，日期用**替换")
        print("5. 银行卡号：保留前4位和后4位，中间用*替换")
        print("6. 其他字段：保持不变")

        print("\n脱敏处理已完成！")
    else:
        print("脱敏处理失败！")

if __name__ == '__main__':
    main()

```

2、运行以上exp.py得到answer.csv，打开看到数据已成功脱敏，结果如下图所示

[illegible]

3、访问验证靶机，提交answer.csv，发现成功通过并且拿到flag。



Drop file here or [click to upload](#)

只允许上传 csv 文件格式 (其中文件名称随意)

[点击此处下载示例图](#)

answer.csv

结果展示区: (请注意, 刷新页面结果将清空)

文件名	上传状态	上传相关备注	分数(百分比)	FLAG
answer.csv	1	上传成功	100.000%	DASCTF{7b43c28f19d5e6a2b0f8c4d7e3a9f156}